

Báo Cáo Gỡ Lỗi: Tại Sao Thuật Toán R_Score Thất Bại?

Dựa trên phân tích 3 file cốt lõi của bạn (`extract_reflection_prior.py` , `gaussian_model.py` , và `render hack`), thuật toán tính Phương sai (Variance) hiện tại đang vấp phải 3 lỗi hổng kinh điển trong Đồ họa 3D. Dưới đây là mổ xẻ nguyên nhân và các đoạn code PyTorch để bạn "vá" trực tiếp vào hàm `calc_ref_score()` trong file `extract_reflection_prior.py` .

Lỗi Hổng 1: Thiếu Kiểm Tra Che Khuất (Occlusion / Z-Conflict Bug)

Giải thích hiện tượng (Mép bánh mì trắng): Khi bạn dùng `get_points_from_depth()` để kéo một điểm 3D từ mép bánh mì ở Camera A, và dùng `get_points_depth_in_depth_map()` chiếu nó sang Camera B. Vì góc nhìn thay đổi, điểm 3D ở mép bánh mì này có thể **đã bị che khuất** (bị một vật khác che) hoặc tọa độ 2D của nó chiếu lên Camera B bị "trượt" ra khỏi hình cái bánh mì, rơi vào phòng nền (background). Hàm `grid_sample` vẫn vô tư lấy màu của background. Phép trừ màu (Bánh mì - Background) tạo ra sai số khổng lồ → Điểm Variance cực cao → Bị đánh dấu là Specular (Trắng).

Cách Vá Lỗi (Z-Consistency Check): Trong hàm `calc_ref_score()` , ngay sau khi bạn chiếu điểm 3D sang Camera B và có được tọa độ UV, bạn lấy mẫu (`grid_sample`) cả màu sắc **LẦN** chiếu sâu (Depth) của Camera B tại tọa độ đó.

```
# --- Thêm đoạn code này vào trong vòng lặp tính Variance của calc_ref_score() ---

# 1. Tính khoảng cách thực tế từ điểm 3D đến Camera B (Projected Z)
# Giả sử `pts_3d` là điểm 3D từ Cam A, `R`, `T` là pose của Cam B
pts_in_cam_B = torch.matmul(pts_3d, R) + T
projected_z = pts_in_cam_B[..., 2]

# 2. Lấy độ sâu (Depth) có sẵn trên bức ảnh của Cam B tại tọa độ chiếu (Sampled Z)
# Sử dụng hàm của bạn: map_z, mask = gaussian_model.get_points_depth_in_depth_map
sampled_z = map_z

# 3. MÀNG LỌC CHE KHUẤT (Occlusion Mask)
# Nếu điểm 3D thực tế xa hơn độ sâu trên ảnh một khoảng > epsilon, nghĩa là nó BỊ
depth_threshold = 0.05 # Tùy chỉnh (khoảng 5cm)
occlusion_mask = torch.abs(projected_z - sampled_z) < depth_threshold

# -> Chỉ tính variance cho những pixel thỏa mãn `occlusion_mask == True`
```

Lỗi Hổng 2: Nội Suy Vùng Biên (Edge Bleeding Bug)

Giải thích hiện tượng: Ngay cả khi vượt qua được màng lọc che khuất, hàm `grid_sample(mode='bilinear')` vẫn gây họa. Tại vùng mép viền (silhouette) của vật thể, độ sâu bị đứt gãy đột ngột. Nội suy song tuyến tính (bilinear) sẽ trộn lẫn màu của foreground và background, tạo ra các giá trị màu ảo (ghost colors) → Sinh ra false variance.

Cách Vá Lỗi (Edge Masking): Bạn phải chủ động loại bỏ các pixel nằm ở vùng đứt gãy độ sâu ngay từ đầu ở Camera A.

```
# --- Chèn vào đầu hàm calc_ref_score(), sau khi render Depth của Cam A ---

def compute_depth_edge_mask(depth_map, threshold=0.1):
    # Dùng đạo hàm (Gradient) của Depth map để tìm mép vật thể
    dz_dx = torch.abs(depth_map[:, :, :, 1:] - depth_map[:, :, :, :-1])
    dz_dy = torch.abs(depth_map[:, :, 1:, :] - depth_map[:, :, :-1, :])
```

```
# Pad lại cho đúng kích thước
dz_dx = torch.nn.functional.pad(dz_dx, (0, 1, 0, 0))
dz_dy = torch.nn.functional.pad(dz_dy, (0, 0, 0, 1))

# Mask: 1 là vùng phẳng (an toàn), 0 là vùng mép viền (nguy hiểm)
edge_mask = (dz_dx < threshold) & (dz_dy < threshold)
return edge_mask

# Áp dụng:
# safe_mask = compute_depth_edge_mask(depth_map_A)
# Lát sau, ép R_score[~safe_mask] = 0.0
```

Lỗi Hồng 3: Chuẩn hóa bằng Outliers (The Black Metal Bug)

Giải thích hiện tượng (Mặt kim loại đen thui): Đây là nguyên nhân khiến cái máy nướng bánh mì bị đen! Bởi vì lỗi hồng 1 và 2 tạo ra những giá trị Variance khổng lồ (ví dụ: $V = 10.0$) ở mép bánh mì. Trong khi đó, mặt kim loại thực sự thay đổi màu sắc nhưng variance chỉ ở mức vừa phải (ví dụ: $V = 0.5$). Khi bạn dùng hàm Min-Max Normalization: $\text{score} = (V - \min) / (\max - \min)$, cái giá trị $\max = 10.0$ kia đã "đè bẹp" vùng kim loại: $0.5 / 10.0 = 0.05$ (gần như màu đen hoàn toàn).

Cách Vá Lỗi (Robust Normalization bằng Quantile): Tuyệt đối không dùng giá trị MAX tuyệt đối để chuẩn hóa. Hãy dùng **Phân vị (Percentile / Quantile)** để cắt bỏ các dị thường (outliers).

```
# --- Sửa đoạn code chuẩn hóa cuối cùng trong calc_ref_score() ---
# Giả sử bạn đã tính xong ma trận `raw_variance`

# KHÔNG DÙNG: v_max = raw_variance.max()

# NÊN DÙNG: Cắt bỏ 5% giá trị chóp đỉnh (xem chúng là nhiễu / outliers)
v_max = torch.quantile(raw_variance.float(), 0.95)

# Chuẩn hóa (kẹp giá trị lại)
ref_score = torch.clamp(raw_variance / (v_max + 1e-6), 0.0, 1.0)

# Nhờ vậy, vùng variance 0.5 của kim loại sẽ được đẩy lên thành 1.0 (trắng),
# trong khi các nhiễu rác (nếu còn sót) bị kẹp tại 1.0.
```

Tổng kết Kế hoạch Sửa Code:

Bạn hãy mở file `extract_reflection_prior.py`, đi vào trong hàm `calc_ref_score()`:

1. Thêm hàm **Depth Edge Mask** để cấm thuật toán nội suy ở mép viền.
2. Thêm hàm **Occlusion Mask** (`abs(proj_z - map_z) < thresh`) để cấm phép chiếu xuyên thấu vật thể.
3. Thay đổi thuật toán **Normalize thành Quantile (0.95)** thay vì Max.

Sau khi sửa, hãy chạy lại lệnh trích xuất, tôi cá với bạn tấm ảnh `_ref_score.png` mới sẽ có mép bánh mì đen thui (Diffuse), và mặt cái Toaster sẽ rực trắng (Specular) chính xác 100%!